Parsa Dalirirad, ID: 40314844
AI Logs - SOEN341 - Sprint 2

Role: Backend Development - Python Flask, Supabase Authentication, PostgreSQL

Task 1: Review Sprint 2 Requirements, Repository Structure, and Development Issues

AI Tool Used: ChatGPT (OpenAI)

Purpose of AI Use: Requirements analysis, repository review, task planning, and GitHub issue preparation.

Prompt: Help organize the backend responsibilities for sprint 2 and give me order on the priorities of each task

AI-Suggested Content: AI helped organize my Sprint 2 backend responsibilities around Issue #41 (Job Application Submission API), Issue #42 (Job Application Tracking API), and Issue #43 (Recruiter Job Posting Backend API). It also helped review the available backend structure and prepare issue descriptions, acceptance criteria, dependencies, and implementation tasks using the team issue template.

Validation: I compared the proposed task breakdown with my assigned Sprint 2 work and the team's GitHub issue format. The uploaded repository archive available for this review is an older baseline and does not contain the final Sprint 2 backend, so it was not treated as proof of the current implementation.

Modifications Made: I narrowed the AI suggestions to my backend responsibilities and kept database/schema ownership separate. Dependencies on Hatem's PostgreSQL schema and recruiter-role configuration were recorded rather than assumed.

Decision: Modified before use.

Reflection: [Suggested wording - review before submission] AI helped me turn the Sprint 2 scope into a clear sequence of backend tasks. I still had to verify responsibilities and dependencies with the team because AI could not determine the current repository state or database ownership by itself.

Evidence: GitHub Issues #41, #42, and #43. ISSUE_TEMPLATE.md.

Responsible Person: Parsa Dalirirad

Task 2: Design and Integrate Issue #41 - Job Application Submission API

AI Tool Used: ChatGPT (OpenAI)

Purpose of AI Use: Backend API design, Flask code generation, validation/error handling, and integration guidance.

Prompt: help me create the job application submission for backend and give me all the head ups neaded in the route of making it.

AI-Suggested Content: AI proposed a Flask implementation for job application submission, including authenticated request handling, input validation, application creation logic, PostgreSQL/Supabase interaction patterns, HTTP error responses, and checks needed before creating an application. AI also helped explain how to integrate the new route into the existing app.py rather than replacing teammates' code.

Validation: The generated code was reviewed as a starting point and compared with the existing Flask structure. I specifically checked that the integration approach preserved existing imports and routes and did not overwrite shared backend work. Syntax verification, automated test results, manual API results, and live Supabase/PostgreSQL integration evidence are not available in the supplied project evidence.

Modifications Made: The standalone AI proposal was adapted to the shared app.py and to the team's existing database/job structures. Any assumed field names or relationships had to follow the actual team schema instead of AI placeholders.

Decision: Modified before use. Final live database validation remains unconfirmed in the available evidence.

Reflection: [Suggested wording - review before submission] AI gave me a useful starting structure for the submission endpoint, but integrating it into a shared backend required more care than generating a new file. I had to preserve teammate code and make the route match the real database and job identifiers.

Evidence: Issue #41 - Implement Job Application Submission API; app.py.

Responsible Person: Parsa Dalirirad

Task 3: Design and Integrate Issue #42 - Job Application Tracking API

AI Tool Used: ChatGPT (OpenAI)

Purpose of AI Use: Backend implementation, authentication/authorization design, application-status handling, and integration review.

Prompt: help me create the job application tracking API for backend and give me all the head ups neaded in the route of making it.

AI-Suggested Content: AI proposed Flask routes and supporting logic for retrieving and tracking job applications. The assistance covered Supabase-authenticated user checks, authorization/ownership checks, application status handling, missing-record/error cases, and integration with the application-submission and latest job-listing backend code.

Validation: I reviewed the proposed behavior for user-specific access and compatibility with the project's Supabase Authentication approach. The integration also required checking the latest job-listing code so shared job identifiers and existing routes were reused instead of duplicated. No supplied evidence confirms automated tests or live database integration.

Modifications Made: Authentication, status handling, database queries, and shared job references were adapted to the existing repository and actual team schema. AI-generated assumptions that conflicted with current code were to be changed rather than accepted.

Decision: Modified before use. Database-backed verification remains pending where no evidence is available.

Reflection: [Suggested wording - review before submission] AI helped me separate authentication, authorization, data ownership, and status logic. I learned that an endpoint can look correct but still be wrong if it exposes another user's application or does not match the current database schema.

Evidence: Issue #42 - Implement Job Application Tracking API; integration with Issue #41 and the latest job-listing backend.

Responsible Person: Parsa Dalirirad

Task 4: Design Issue #43 - Recruiter Job Posting Backend API

AI Tool Used: ChatGPT (OpenAI)

Purpose of AI Use: Backend API design, Flask code generation, authentication/authorization, validation, and database integration planning.

Prompt: help me create the job posting backend API for backend and give me all the head ups neaded in the route of making it.

AI-Suggested Content: AI proposed recruiter-facing job-posting backend logic, including authenticated route handling, recruiter authorization checks, input validation, job creation/update operations, database interactions, and structured error responses. The proposed design was intended to integrate with the existing job-related backend rather than create a separate system.

Validation: I reviewed the proposal against the project's Supabase-authenticated backend approach. Live recruiter authorization and PostgreSQL behavior could not be treated as verified without Hatem's final schema and recruiter-role configuration. No automated-test or live integration result is available in the supplied evidence.

Modifications Made: Any AI-assumed recruiter field, role table, schema relationship, or database column had to be replaced with the actual team configuration. The generated routes were adapted to the current Flask/job code.

Decision: Modified before use; live recruiter-role/database integration validation was deferred where the required shared configuration was not yet verified.

Reflection: [Suggested wording - review before submission] AI helped me structure the recruiter authorization and job-posting logic, but this task made the dependency on team database design very clear. Backend code could only be considered complete after it matched the actual role and schema configuration.

Evidence: Issue #43 - Implement Recruiter Job Posting Backend API. Dependency: Hatem's schema and recruiter-role configuration.

Responsible Person: Parsa Dalirirad

Task 5: Prepare Pull Request Descriptions for Issues #41, #42, and #43

AI Tool Used: ChatGPT (OpenAI)

Purpose of AI Use: Technical documentation and GitHub pull-request preparation.

Prompt: Help me prepare the pull request description by using the template provided so I can fill up the needed parts on it.

AI-Suggested Content: AI helped structure PR descriptions for the three backend issues, including the related issue, summary of backend changes, authentication/database considerations, testing status, documentation notes, and unresolved dependencies.

Validation: The PR wording was intended to be compared with the actual branch diff before submission. The team PULL_REQUEST_TEMPLATE.md requires related issue, summary, changes made, testing performed, documentation, AI-assistance recording, and readiness for review. PR URLs, approvals, merge status, and verified test results are not available in the supplied evidence.

Modifications Made: The generated descriptions had to be edited so they described only changes actually present in each branch. Testing checkboxes should only be marked complete when corresponding evidence exists.

Decision: Modified before use.

Reflection: [Suggested wording - review before submission] AI helped me write clearer PR descriptions, but I remained responsible for making sure the text matched the actual code changes and did not overstate testing, database integration, or completion status.

Evidence: PULL_REQUEST_TEMPLATE.md; PRs for Issues #41, #42, and #43. PR URLs and merge/approval evidence: Not provided - to be completed by student.

Responsible Person: Parsa Dalirirad

Sprint 2 Verification Notes

AI-Generated Work Summary: AI assisted with planning, GitHub issue preparation, Flask API proposals for Issues #41-#43, integration review, merge-conflict troubleshooting, candidate unit tests, API documentation, validation planning, and PR descriptions.

Human Oversight: I remained responsible for reviewing and integrating generated code, preserving teammates' work, checking authentication/authorization behavior, matching the actual PostgreSQL schema, resolving Git conflicts, running tests, validating live Supabase/database behavior, and deciding what was submitted.

Known Limitations: Automated test success, manual API success, live Supabase/PostgreSQL integration, PR approval/merge status, commit hashes, exact historical dates, and exact historical prompts are not claimed unless evidence is added by the student. Hatem's schema and recruiter-role configuration are explicit dependencies for database-backed and recruiter-only functionality.